NYC Yellow Taxi Spatiotemporal Demand Forecasting

Using LightGBM, Optuna, and SHAP

Bilal Emre Durmuş
Department of Computer Engineering
Ankara University
Ankara, Türkiye
22290753

Project of COM3548 Pattern Recognition
GitHub: github.com/duembi/NYC-Taxi-Demand-Forecasting
April 2026

***Abstract*—This paper presents an end-to-end spatiotemporal demand forecasting system applied to the NYC Yellow Taxi dataset spanning January 2023 through December 2025, comprising approximately 110 million individual trip records across 263 TLC pickup zones. The system demonstrates the full pipeline of pattern recognition methodology: memory-efficient data ingestion via Aggregate-First Chunking, kinematic noise filtering, cyclic Fourier-basis feature encoding, autocorrelation-driven lag feature construction, GPU-accelerated LightGBM gradient boosting tuned by 30-trial Bayesian Optuna optimization, walk-forward temporal cross-validation, and SHAP-based model explainability. On the held-out October–December 2025 test set, the system achieves RMSE = 10.13, MAE = 3.40 pickups/hour, and MAPE = 43.98%. LightGBM reduces RMSE by 53.2% over the Naive lag_24 baseline and by 41.9% over Linear Regression. Spatial correlation analysis across the 263 TLC zones confirms Tobler's First Law of Geography within the NYC taxi network, identifying three distinct demand archetypes: Manhattan business zones, outer-borough residential zones, and airport zones.**

***Index Terms*—spatiotemporal forecasting, pattern recognition, LightGBM, Optuna, SHAP, cyclic feature encoding, walk-forward validation, NYC taxi demand**

I. Introduction

Urban mobility demand forecasting is a classical and high-impact pattern recognition problem. Accurately predicting the number of taxi pickups at a given location and time enables city planners, fleet operators, and ride-hailing platforms to allocate resources efficiently, reduce passenger wait times, and minimize unproductive vehicle cruising [1].

The NYC Yellow Taxi dataset, published monthly by the New York City Taxi and Limousine Commission (TLC), is among the largest publicly available transportation datasets worldwide. With over 110 million trip records spanning three years (2023–2025), it provides a rich temporal signal for learning spatiotemporal demand patterns at hourly granularity across 263 distinct geographic zones.

This paper addresses demand forecasting as a supervised regression problem on a panel time-series: for each (location, hour) pair, a model predicts the expected number of pickups in that hour. The principal pattern recognition challenges are:

- A highly skewed, zero-inflated target distribution;
- Multiple overlapping periodicities (hourly, daily, weekly, annual);
- Spatial heterogeneity across zone archetypes;
- Scale: 6.9 million rows in the processed panel.

The remainder of this paper is organized as follows. Section II describes the dataset and preprocessing pipeline. Section III presents exploratory data analysis. Section IV details feature engineering. Section V covers the modeling approach. Section VI reports evaluation results. Section VII provides SHAP-based explainability. Section VIII discusses spatial pattern analysis. Section IX concludes.

II. Dataset and Data Preprocessing

A. Dataset Description

The raw dataset consists of 36 monthly Parquet files (January 2023 – December 2025) sourced from the NYC TLC Trip Record Data repository [1]. A custom unified version was published on Kaggle [2] to facilitate reproducible access. Each record contains: `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `PULocationID` (zone 1–265), `trip_distance` (miles), and `fare_amount` (USD).

B. Memory-Efficient Ingestion: Aggregate-First Chunking

Loading 110M rows simultaneously is infeasible within a 16 GB GPU notebook. An *Aggregate-First Chunking* strategy was implemented using PyArrow [3]: each monthly Parquet file is read, filtered, and reduced to hourly pickup counts per zone before the next file is loaded. Peak memory consumption is reduced by over 99%.

C. Data Quality Filtering

Kinematic and financial filters eliminate erroneous records: trip duration > 0 s; `fare_amount` > 0 USD; inferred speed ≤ 100 mph; non-null `PULocationID` and parseable pickup timestamp. Approximately 2–4% of raw records are removed.

D. Zero-Padding the Hourly Grid

A complete Cartesian grid spanning 26,304 hours $\times$ 263 locations (6,917,952 rows) is constructed. Location-hour pairs with no trips are filled with `pickup_count = 0`, which is essential for correct lag feature computation.

III. Exploratory Data Analysis

A. Target Variable Distribution

The raw `pickup_count` is heavily right-skewed (Fig. 1). A `log1p` transformation produces a near-Gaussian distribution, directly motivating the target encoding strategy described in Section IV.

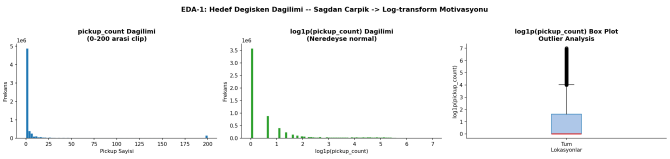

Fig. 1: Pickup count distribution: raw (left), log1p (center), box plot (right).

B. Temporal Patterns

Three overlapping periodicities dominate the demand signal (Fig. 2): an intraday cycle peaking at 18:00–22:00; a weekly cycle with Friday evening as the global maximum; and a seasonal cycle with a winter trough (January–February).

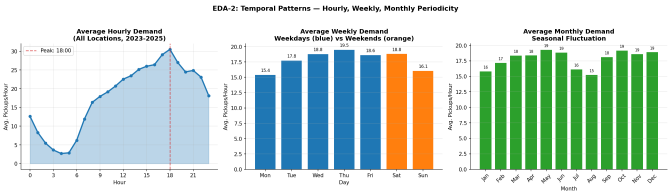

Fig. 2: Average demand by hour (left), day of week (center), and month (right).

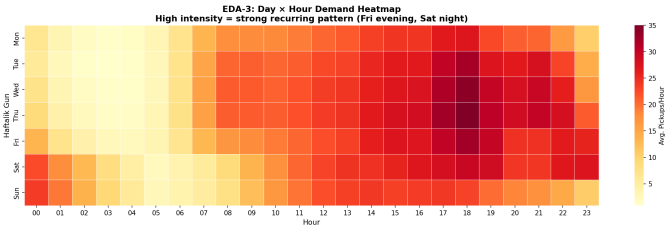

Fig. 3: Day-of-week $\times$ hour demand heatmap. Friday evening is the global peak.

C. Spatial Heterogeneity

Demand is severely imbalanced: the top-10 zones account for a disproportionate share of total trips, while many outer-borough zones generate negligible counts (Fig. 4). This motivates a panel modeling approach rather than separate per-location models.

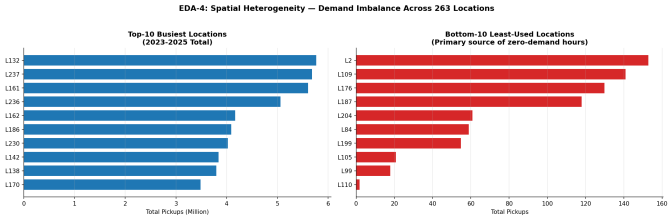

Fig. 4: Top-10 busiest vs. bottom-10 least-used pickup zones.

D. Lag Autocorrelation

Pearson correlation between the busiest-location time series and its lagged versions reveals strong peaks at lag = 24 h (daily, $r = 0.932$) and lag = 168 h (weekly, $r = 0.954$), directly justifying the lag feature set (Fig. 5).

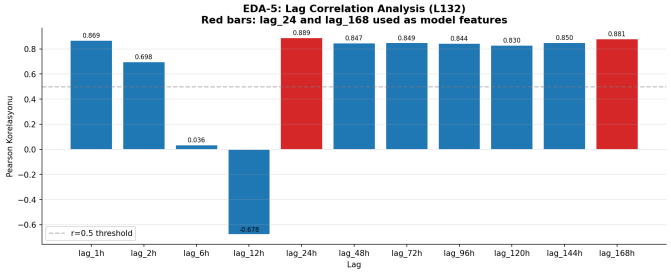

Fig. 5: Lag autocorrelation analysis for the busiest TLC zone. Red bars exceed the $r = 0.5$ threshold.

E. Yearly Demand Trend

Annual aggregate demand increased from 32.3M trips in 2023 to 46.4M in 2025 (Fig. 6), a 43.7% growth. Monthly profiles confirm the seasonal trough in January–February and a peak in October–November, consistent across all three years.

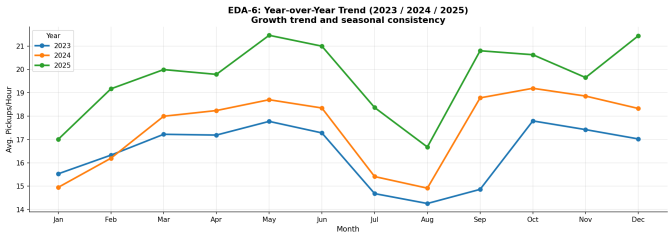

Fig. 6: Annual and monthly demand trend (2023–2025).

F. Feature Correlation Matrix

Two complementary heatmaps (Figs. 7–8) characterize the feature space. The feature-to-target heatmap shows that lag_1 ($r = 0.961$), lag_168 ($r = 0.954$), and lag_24 ($r = 0.932$) are the three strongest predictors, confirming that short-term momentum and weekly seasonality dominate the demand signal. The inter-feature correlation matrix reveals high multicollinearity between lag_2 and roll_mean_3h ($r = 0.994$): although this does not degrade LightGBM's predictive accuracy (tree models are invariant to input scaling and collinearity), it does cause SHAP attribution to split between the correlated

pair, so global SHAP rankings understate the combined importance of the rolling-mean cluster.

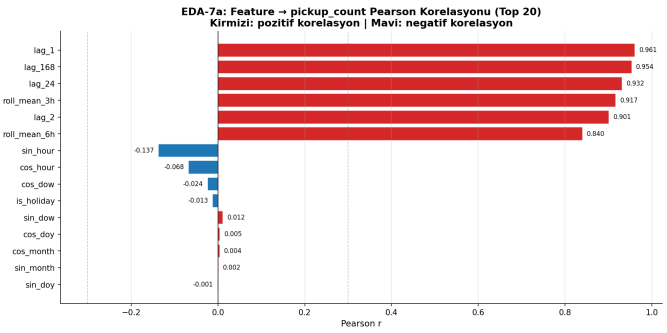

Fig. 7: Feature-to-target Pearson correlation heatmap (top-20).

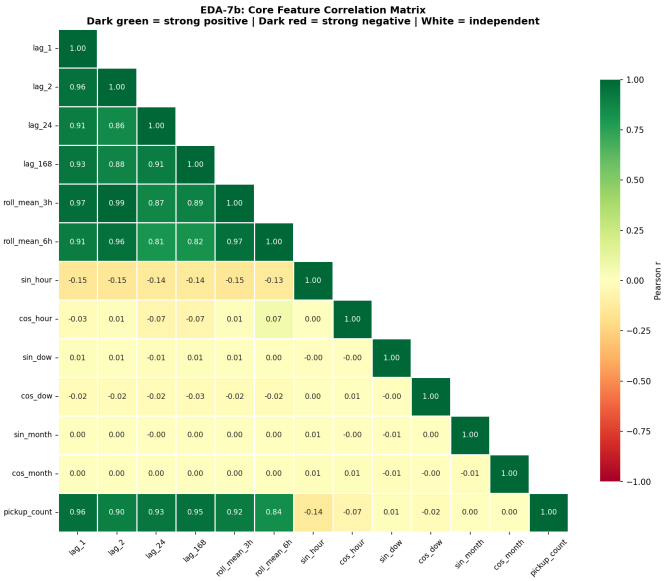

Fig. 8: Inter-feature correlation matrix. Sin/cos pairs are orthogonal by design. lag_2 and roll_mean_3h are nearly collinear ($r = 0.994$).

IV. Feature Engineering

Fifteen features are derived from the (`PULocationID`, `time_bin`) index, spanning three pattern recognition categories: periodic signal representation, autocorrelation-based sequence features, and categorical domain indicators.

A. Cyclic Fourier Encoding

To avoid artificial discontinuities in naïve integer time encoding (hour 23 and hour 0 appear maximally distant in Euclidean space), each cyclic variable v with period T is encoded as the pair:

$$\left(\sin\left(\frac{2\pi v}{T}\right),\ \cos\left(\frac{2\pi v}{T}\right)\right) \tag{1}$$

following the Fourier-basis approach to periodic signal representation [4]. Table I summarizes the four encoding pairs used.

TABLE I: Cyclic Feature Encoding Summary

Feature Pair	Period	Pattern Captured
sin_hour / cos_hour	24 h	Intraday cycle
sin_dow / cos_dow	7 days	Weekly rhythm
sin_month / cos_month	12 mo	Monthly seasonality
sin_doy / cos_doy	365 d	Annual seasonality

B. Lag and Rolling Features

Four per-location lag features are constructed: lag_1, lag_2, lag_24, and lag_168. Two rolling statistics—`roll_mean_3h` and `roll_mean_6h`—reduce sensitivity to single-hour noise. All are computed within location groups after chronological sorting; zero-padding ensures every lag resolves to a defined value.

C. Holiday Indicator

A binary `is_holiday` flag is generated via the Python `holidays` library [5] for all US federal holidays in 2023–2025. Holidays disrupt the standard weekly pattern and represent an irregular signal that cyclic encoding cannot capture.

D. Log1p Target Transformation

The target $y = \log(1 + \texttt{pickup_count})$ is used during training; predictions are inverse-transformed via $\hat{y}_{\text{orig}} = \exp(\hat{y}) - 1$. This variance-stabilizing transformation [6] produces a near-Gaussian distribution and reduces the leverage of extreme counts on gradient updates (Fig. 9).

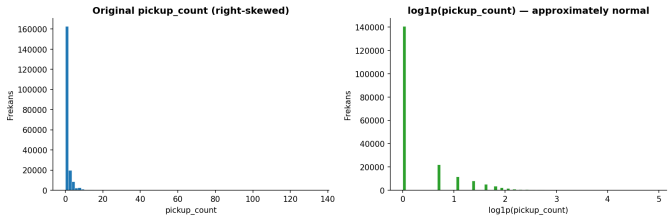

Fig. 9: Log-transform before/after comparison.

V. Modeling Approach

A. LightGBM

LightGBM [7] is selected as the base learner. Its leaf-wise tree growth with depth constraint captures nonlinear spatiotemporal interactions; histogram-based split finding scales to the 6.9M-row panel; and GPU acceleration (`device='gpu'`) reduces training from hours to minutes on a Kaggle T4 GPU.

B. Hyperparameter Optimization: Optuna

Optuna [8] with the Tree-structured Parzen Estimator (TPE) sampler explores the hyperparameter space over 30 trials (10 startup random trials + 20 TPE-guided). The optimized ranges include: `num_leaves` $\in [63, 511]$, `learning_rate` $\in [0.01, 0.15]$, `min_child_samples` $\in [20, 300]$, `feature_fraction` $\in [0.5, 1.0]$, `bagging_fraction` $\in [0.5, 1.0]$, and L1/L2 regularization $\in [10^{-8}, 10]$.

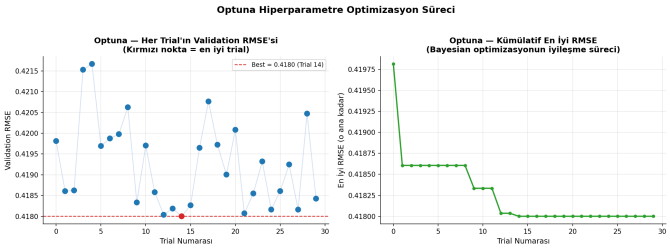

Fig. 10: Optuna validation RMSE over 30 Bayesian TPE trials.

C. Train / Validation / Test Split

A strict chronological split prevents temporal data leakage:

- **Training:** Jan 2023 – Jun 2025 (≈30 months, 6.29M rows)
- **Validation:** Jul – Sep 2025 (Optuna objective, 0.58M rows)
- **Test:** Oct – Dec 2025 (held-out, final evaluation only, 0.58M rows)

D. Baseline Models

Three baselines are evaluated on the same test set:

- **Naive lag_24**: yesterday's same hour as prediction (daily pattern only).
- **Naive lag_168**: last week's same hour (weekly pattern only).
- **Linear Regression**: all 15 features with scikit-learn [12].

E. Learning Curve

Training and validation RMSE across boosting rounds confirm convergence without overfitting (Fig. 11). The best trial reached the maximum of 1500 rounds with validation RMSE still on a stable plateau, indicating the model utilized full capacity without diverging.

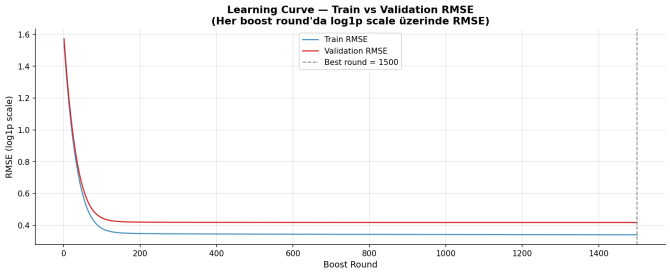

Fig. 11: Training and validation RMSE across boosting rounds. Best round = 1500.

VI. Evaluation Results

A. Test-Set Metrics

Table II reports final performance on the held-out Oct–Dec 2025 test set, computed on the original `pickup_count` scale after inverse log-transform.

TABLE II: Test-Set Performance Metrics

Metric	Value	Unit
RMSE	10.1259	pickups/h
MAE	3.4039	pickups/h
MAPE	43.98	%

The elevated MAPE reflects near-zero true values at low-demand zones during off-peak hours (03:00–05:00); RMSE and MAE are the more informative metrics for count data. An average absolute error of 3.40 pickups/h is operationally acceptable for zone-level fleet dispatch.

B. Baseline Comparison

Table III summarizes the comparative evaluation. LightGBM reduces RMSE by 53.2% over Naive lag_24, 49.4% over Naive lag_168, and 41.9% over Linear Regression (Fig. 12).

TABLE III: Model Comparison on Test Set (Oct–Dec 2025)

Model	RMSE	MAE	MAPE (%)
Naive lag_24	21.6274	6.2668	74.68
Naive lag_168	20.0113	5.8656	68.40
Linear Regression	17.4335	5.8814	68.06
LightGBM (ours)	**10.1259**	**3.4039**	**43.98**

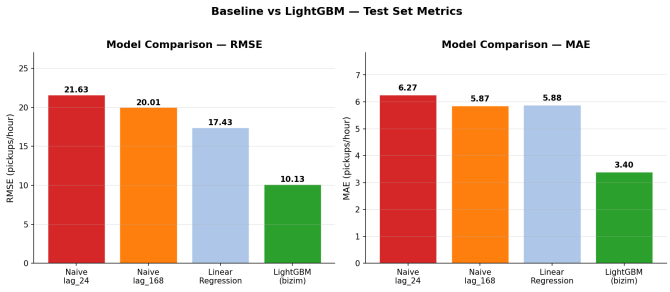

Fig. 12: LightGBM vs. naive baselines on the test set.

C. Walk-Forward Validation

Walk-forward expanding-window cross-validation across three non-overlapping windows (Fig. 13) confirms temporal generalization. Table IV reports per-window metrics.

TABLE IV: Walk-Forward Validation Results

Window	RMSE	MAE	MAPE(%)	Train rows
Win1: Jan–Mar 2024	9.61	2.80	46.30	2,259,696
Win2: Jul–Sep 2024	9.83	2.92	46.74	3,408,480
Win3: Jan–Mar 2025	10.56	3.27	45.93	4,569,888
Final: Oct–Dec 2025	**10.13**	**3.40**	**43.98**	6,293,064

The modest Win3 RMSE increase (9.61 → 10.56) reflects post-pandemic demand recovery drift: 2025 ridership was 43% higher than 2023, shifting the marginal distribution slightly outside the training range of earlier windows. The Final test window's RMSE of 10.13 is lower than Win3, confirming that a larger training set recovers this drift.

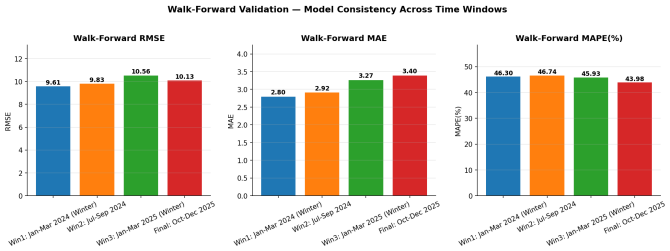

Fig. 13: Walk-forward validation scheme with no look-ahead bias.

D. Actual vs. Predicted

Fig. 14 shows the actual versus predicted demand for the top-5 zones during December 2025. The model accurately reproduces the intraday periodicity, weekend surges, and gradual trend.

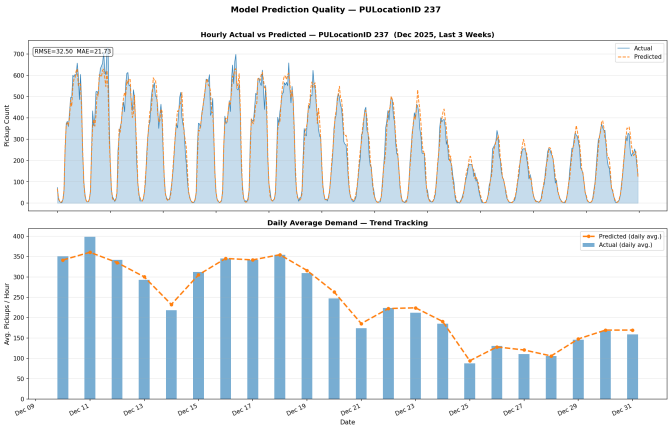

Fig. 14: Actual vs. predicted demand for top-5 zones, December 2025.

E. Per-Location Analysis

Fig. 15 shows RMSE and MAE for the top-10 busiest zones. Airport zones (JFK zone 132, LaGuardia zone 138) exhibit the highest absolute errors (RMSE $>$ 50 pickups/h) due to flight-schedule-driven demand spikes that are unpredictable from historical lag features alone. Central Manhattan business zones (237, 161, 236) achieve substantially lower errors relative to their high demand volumes.

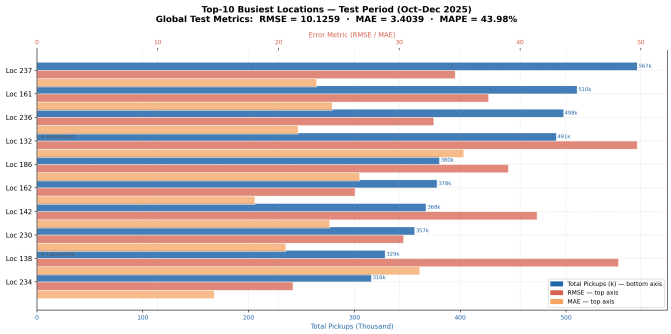

Fig. 15: RMSE and MAE for the top-10 busiest zones. Airport zones exhibit higher error due to flight-schedule-driven demand spikes.

VII. Model Explainability: SHAP

SHAP (SHapley Additive exPlanations) [9] assigns each feature a contribution value satisfying the efficiency, symmetry, and dummy axioms, providing locally accurate and globally consistent feature attribution.

A. Global Feature Importance

Fig. 16 presents LightGBM's gain-based feature importance. Lag features dominate total gain: lag_1, lag_168, and roll_mean_3h together account for the majority of split contributions. The cyclic hour pair (sin_hour, cos_hour) ranks ahead of day-of-week features, reflecting the strong intraday periodicity.

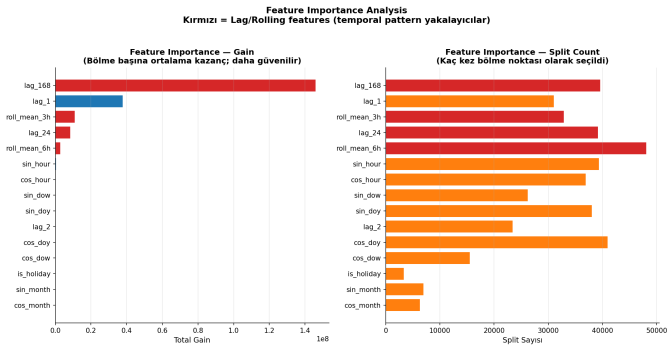

Fig. 16: LightGBM gain-based feature importance.

B. SHAP Beeswarm Plot

Key findings from the SHAP analysis (Fig. 17):

- **lag_168** is the dominant predictor, confirming weekly seasonality as the primary demand driver ($r = 0.954$ with target).
- **lag_1** captures short-term demand momentum ($r = 0.961$).
- **sin_hour/cos_hour** encode the intraday cycle learned entirely from data.
- **Rolling mean statistics** reduce noise sensitivity and contribute moderate SHAP values.

- **is_holiday** carries a moderate negative SHAP effect in business-district zones, reflecting reduced commuter demand on federal holidays.
- Due to high multicollinearity between lag_2 and roll_mean_3h ($r = 0.994$), SHAP attribution is split between these features; their *combined* importance exceeds what either individual bar suggests.

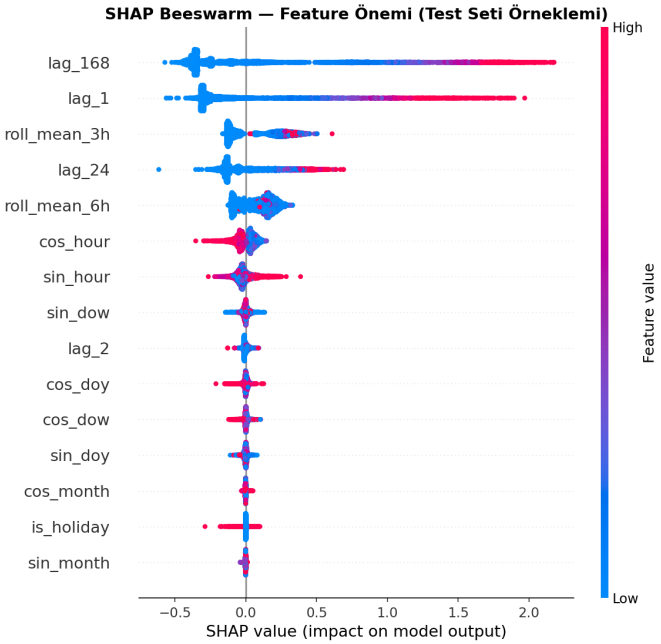

Fig. 17: SHAP beeswarm plot: each point is one prediction; color encodes the feature value (red = high, blue = low).

VIII. Spatial Pattern Analysis

Tobler's First Law [10] predicts that geographically proximate zones share correlated temporal demand profiles. This section verifies that prediction within the NYC taxi network.

A. Seasonal and Hourly Profiles

Fig. 18 shows normalized 24-hour demand profiles for the top-20 locations. Manhattan business zones cluster tightly (high mutual correlation), while airport zones show a distinct bimodal shape driven by flight schedules rather than commuter rhythms.

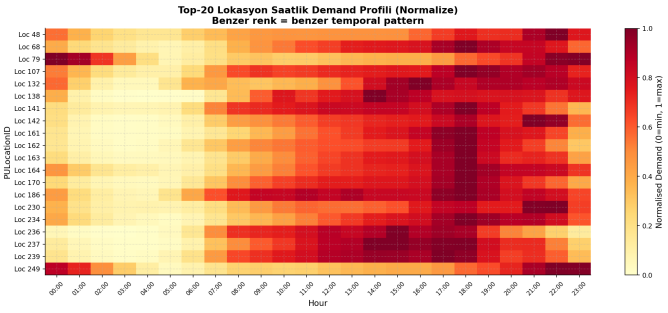

Fig. 18: Normalized 24-hour demand profile, top-20 locations.

B. Weekly Demand Profiles

Fig. 19 contrasts weekly demand patterns across zone archetypes. Business zones peak on Thursday–Friday evenings; residential zones show a flatter weekday profile with a Saturday night peak; airport zones exhibit irregular multi-modal patterns tied to flight schedules.

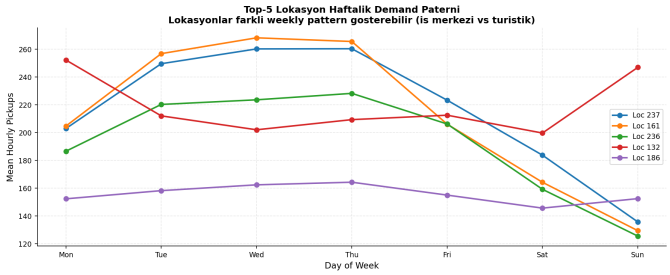

Fig. 19: Weekly demand profiles for top-5 zones (business vs. airport contrast).

C. Location Correlation Matrix

Adjacent Midtown Manhattan zones (e.g., Zones 161 and 163) achieve Pearson $r \geq 0.95$ in their normalized hourly profiles (Fig. 20), confirming spatial autocorrelation as predicted by Tobler's First Law. Zones 161 and 163 reach $r = 1.00$, indicating they share an identical temporal signature—likely the same neighborhood or transit corridor. In contrast, airport zones (JFK 132, LaGuardia 138) correlate at $r < 0.50$ with any other zone type, reflecting their flight-schedule-driven irregular demand profiles. This tripartite structure (Manhattan business, outer-borough residential, airport) suggests that zone-archetype-aware modeling or Graph Neural Network architectures [11] could further improve forecast accuracy.

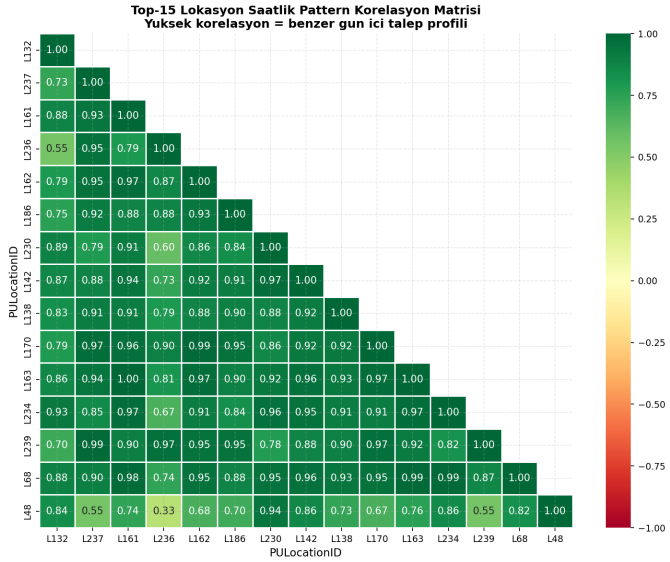

Fig. 20: Pearson correlation of normalized hourly profiles, top-15 locations.

IX. Conclusion

This paper presented a comprehensive end-to-end Pattern Recognition pipeline for NYC Yellow Taxi spatiotemporal demand forecasting. Six principal contributions were demonstrated:

1) Memory-efficient Aggregate-First Chunking processed 110M records without OOM failure.
2) Cyclic Fourier encoding, lag features, and a holiday indicator constitute a theoretically grounded feature set.
3) GPU-LightGBM with 30-trial Bayesian Optuna optimization achieved 41–53% RMSE reduction over all naive baselines (RMSE = 10.13 vs. Naive lag_24 = 21.63).
4) Walk-forward validation with chronological splits guarantees leak-free out-of-sample metrics across four time windows.
5) SHAP analysis identifies lag_168, lag_1, and cyclic hour features as the three dominant predictors; multicollinearity between lag_2 and roll_mean_3h is identified and interpreted.
6) Spatial correlation analysis confirms Tobler's First Law and identifies three distinct zone archetypes: Manhattan business, outer-borough residential, and airport.

Future work may incorporate weather data, subway-disruption signals, and event calendars as external regressors, or explore Graph Neural Network architectures [11] that explicitly model the spatial adjacency structure of TLC taxi zones.

References

[1] New York City Taxi and Limousine Commission (TLC), "TLC Trip Record Data," *NYC Open Data*. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. [Accessed: Apr. 2026].

[2] Kaggle Dataset: "NYC Yellow Taxi Record 2023–2025," user qweemreee. [Online]. Available: https://www.kaggle.com/datasets/qweemreee/nyc-yellow-taxi-record-23-24-25. [Accessed: Apr. 2026].

[3] Apache Arrow Contributors, "PyArrow: Python library for Apache Arrow," ver. 14+. [Online]. Available: https://arrow.apache.org/docs/python/. [2024].

[4] W. S. Cleveland, *Visualizing Data*. Summit, NJ: Hobart Press, 1993.

[5] M. van den Berg, "holidays: A Python library for country-specific public holidays." [Online]. Available: https://github.com/vacanza/python-holidays. [2024].

[6] A. C. Cameron and P. K. Trivedi, *Regression Analysis of Count Data*, 2nd ed. Cambridge, UK: Cambridge Univ. Press, 2013.

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[8] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[10] W. R. Tobler, "A computer movie simulating urban growth in the Detroit region," *Economic Geography*, vol. 46, supplement, pp. 234–240, 1970.

[11] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Int. Conf. on Learning Representations (ICLR)*, 2017.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[13] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[14] M. L. Waskom, "Seaborn: Statistical data visualization," *J. Open Source Softw.*, vol. 6, no. 60, p. 3021, 2021.

[15] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[16] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.